

Node.js · class notes

HTTP Standards & REST APIs

~2 hours · keep asking "WHY?"

★ = exam important ⚡ = interview favourite ✍ = copy in your notes ✗ = common mistake

1 What is HTTP?

The **language** browsers and servers use to talk to each other.

(H) yper

(T) ext (text with links)

(T) ransfer (moving it)

(P) rotocol (rules!)

✎ born 1991, CERN – facts later!

Why does it exist?

Mac + Chrome: "01x@?!"

??

Linux server: "#%&\$"

fix →

both agree to speak HTTP ✓

like two strangers agreeing to speak English

Protocol = a rule book



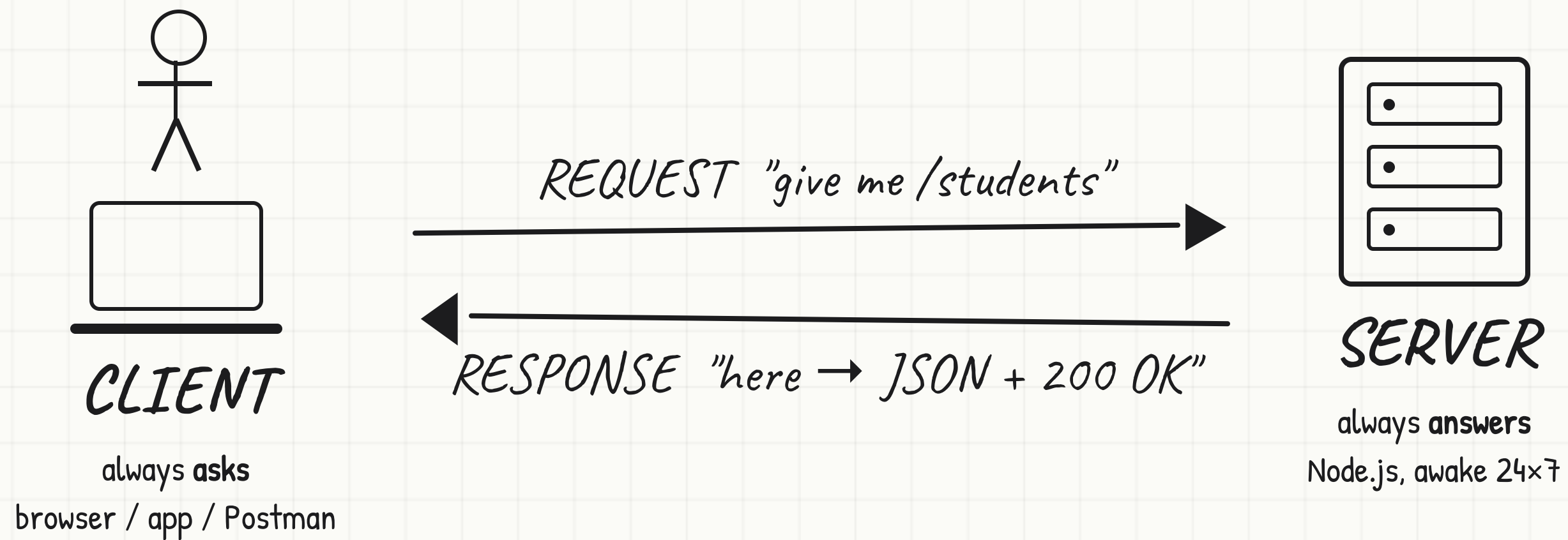
✓ how a message is formatted

✓ which methods are allowed (GET, POST...)

✓ how to report result (status codes)

↳ "HTTP is a protocol" = only **RULES**, not a program – like traffic rules!

① Client & Server



client = customer

server = shopkeeper

cycle: ask → think → answer → forget

★ the server NEVER starts the conversation – it only replies

① A Stateless Protocol ★

The server forgets you after every request!

① "Hi, I am Riya – log me in" → "Welcome!"

② "It's me again!" → "Who are you??"

memory trick: HTTP has goldfish memory

so how do apps stay logged in?

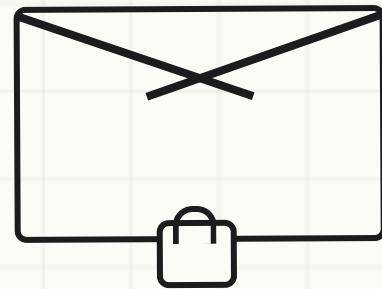


- every request carries its own identity
- a **cookie** or a **token (JWT)** rides along in the headers
- the server checks the pass, not its memory
- 📌 interview: "HTTP is stateless – how do sessions work then?" – THIS is the answer.

① HTTP vs HTTPS (S = Secure) ↴

hi! my
password is
1234...

postcard — anyone reads ✗

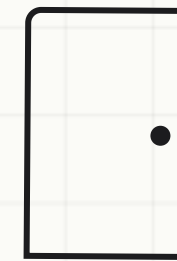


sealed + locked ✓

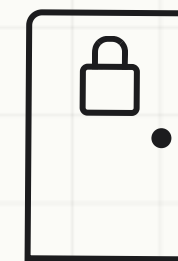
	HTTP	HTTPS
security	none — plain text	TLS encryption
port	80	443
use for logins?	NEVER ✗	always ✓

Port = numbered door

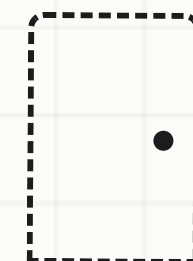
one computer, many doors — each service has its own



:80 http



:443 https



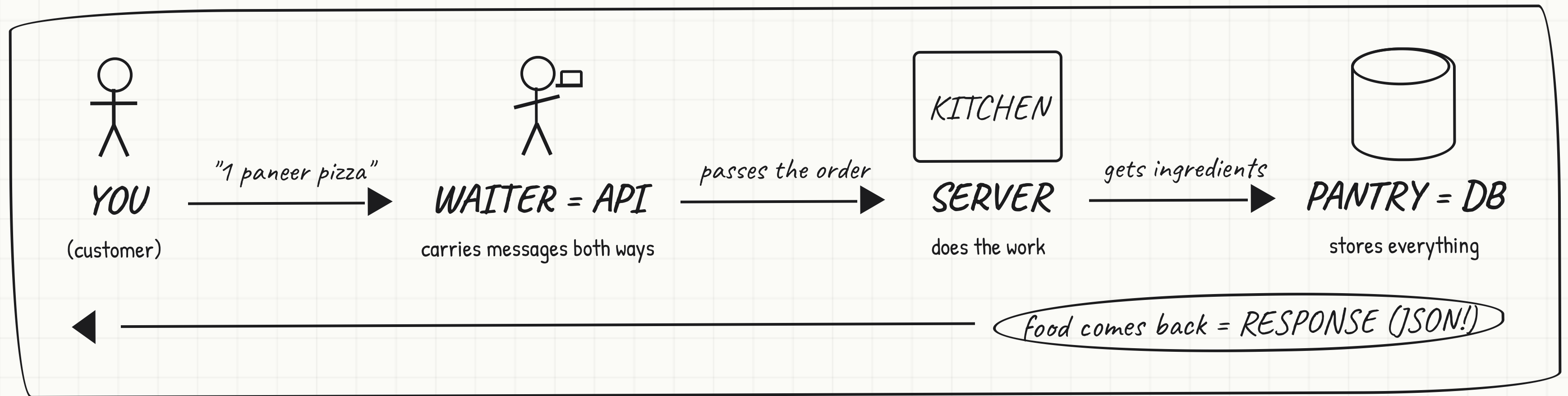
:3000 dev!

🔗 https://x.com is really https://x.com:443 — browser hides it

↴ Node dev servers love :3000 and :8080

2 What is an API?

Application Programming Interface — a **middleman** that lets two programs talk. You never touch the database directly, ever!



★ *you never enter the kitchen — the waiter (API) protects it!*

② The Round Trip



data flows back up as JSON ✓

✎ this same trip happens for EVERY tap you make in ANY app

2 APIs Around You

Google Maps

app asks Maps API → "where am I? fastest route?"

Instagram

feed API fetches posts; like button = POST /likes

WhatsApp

message API delivers your text + the ✓✓ receipts

YouTube

video API sends the list, another streams the video

Paytm / UPI

UPI = APIs between banks! scan → pay → bank APIs settle it

Spotify

search API finds the song; play = stream API

Netflix

recommendation API decides your homepage rows

Weather app

GET api.weather.com/pune → { "temp": 31 }

note

you use 100s of APIs before breakfast — without knowing!

3 Resources – the "things"

A resource = any NOUN your app cares about:

Student Teacher Product Employee
Order Book Customer Course
Payment...

✎ if you can put "a / an / the" before it → it can be a resource

REST resource = a noun with its own URL ★

all students → /students

ONE student, roll 99 → /students/99

their courses → /students/99/courses

URL says **WHICH** thing +

method says **WHAT** to do

↪ nouns in URL, verbs in method – never /getStudents!

④ CRUD – only 4 things ever happen to data

① **Create** – make new

② **Read** – look at it

③ **Update** – change it

④ **Delete** – remove it

CRUD ↔ HTTP ↔ SQL ↴

CRUD	HTTP	SQL	example
Create	POST	INSERT	POST /students – new admission
Read	GET	SELECT	GET /students/99
Update	PUT / PATCH	UPDATE	PATCH /students/99 – fix marks
Delete	DELETE	DELETE	DELETE /students/99 – TC issued!

★ EVERY app – Instagram to ISRO – is just CRUD on resources, dressed up nicely.

4 CRUD Everywhere

College

C — admit a student
R — view student list
U — update marks
D — remove student

Library

C — add new book
R — search catalogue
U — mark "issued"
D — remove torn book

Hospital

C — register patient
R — read reports
U — update prescription
D — discharge record

Amazon

C — place order
R — view orders
U — change address
D — cancel order

Instagram

C — new post
R — scroll feed
U — edit caption
D — delete post

memory trick: Create Posts — Read Gets — Update Puts — Delete Deletes

GET = "GIVE me data"

read-only

purpose: fetch data. Never changes anything!

analogy: reading a library book — look, don't write in it

when: lists, details, search results, feeds

```
GET /students/99 HTTP/1.1
Host: college.com
```

URL: https://college.com/students/99

✗ mistakes: sending a body with GET; using GET to change data (crawlers will click your "delete" links!)

Express

```
app.get('/students/:id', (req, res) => {
  const s = db.find(req.params.id);
  res.json(s); // 200 OK
});
```

response JSON

```
{ "id": 99, "name": "Riya", "course": "BCA" }
← 200 OK
```

trick: GET = GIVE

POST = "PUSH something new"

creates! ✉

purpose: create a NEW resource. Server assigns the id.

analogy: dropping a filled admission form in the college box

when: signup, new order, new post, send message

POST /students HTTP/1.1

Content-Type: application/json

```
{ "name": "Aman", "course": "BCA" }
```

URL: /students — the collection, no id yet!

✗ mistakes: POSTing to /students/99 to create; forgetting Content-Type; expecting POST to be idempotent (2 clicks = 2 orders!)

Express

```
app.post('/students', (req, res) => {  
  const s = db.create(req.body);  
  res.status(201).json(s);  
});
```

response JSON

```
{ "id": 100, "name": "Aman" }  
← 201 Created (not 200!)
```

*trick: POST = POSTbox — new letter
every time*

PUT = "REPLACE the whole thing"

full swap 5

purpose: replace the resource ENTIRELY with what you send

analogy: swapping the whole SIM card — old one is gone

when: "save profile" forms that send every field

```
PUT /students/99 HTTP/1.1
Content-Type: application/json
```

```
{ "name": "Riya S", "course": "MCA", "year": 2 }
```

URL: /students/99 — id required!

X mistakes: sending only 1 field with PUT — the rest become empty/null! (that job is PATCH's)

Express

```
app.put('/students/:id', (req, res) => {
  db.replace(req.params.id, req.body);
  res.json(req.body); // 200
});
```

idempotent ✓

PUT the same body 10 times → same result. Safe to retry!

trick: PUT = PUT a new one in its place

PATCH = "fix just this PART"

purpose: update SOME fields, leave the rest untouched

analogy: puncture repair — patch the hole, keep the tyre

when: change password only, edit caption, mark as read

```
PATCH /students/99 HTTP/1.1
Content-Type: application/json
```

```
{ "course": "MCA" } ← only this changes
```

URL: /students/99

✗ mistakes: using PUT when you mean PATCH (wipes fields!); assuming PATCH is always idempotent (usually yes, not guaranteed)

partial ✂

Express

```
app.patch('/students/:id', (req, res) => {
  const s = db.merge(req.params.id, req.body);
  res.json(s); // 200
});
```

PUT vs PATCH ↴

PUT → repaint the whole wall

PATCH → touch up one spot

trick: PATCH = a patch on jeans

DELETE = "DESTROY it"

purpose: remove the resource at that URL

analogy: shredding one file from the cabinet

when: delete post, cancel booking, remove cart item

```
DELETE /students/99 HTTP/1.1
Authorization: Bearer <token>
```

URL: /students/99 — usually no body at all

✗ mistakes: DELETE /students (boom — whole collection!); no auth check; returning 200 + body when 204 fits better

gone!

Express

```
app.delete('/students/:id', (req, res) => {
  db.remove(req.params.id);
  res.status(204).end(); // No Content
});
```

idempotent ✓

delete twice → still gone. 2nd call just returns 404.

trick: DELETE = DESTROY

5 Method Cheat Table ★ ↴

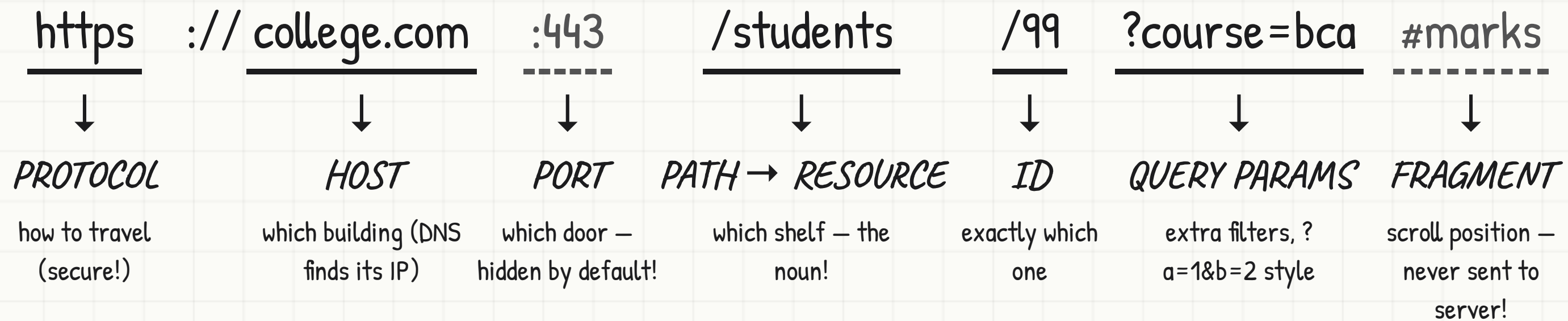
safe = never changes data • idempotent = repeat → same result

<i>method</i>	<i>safe?</i>	<i>idempotent?</i>	<i>body?</i>	<i>success code</i>
GET	✓	✓	✗	200
POST	✗	✗ !!	✓	201
PUT	✗	✓	✓	200
PATCH	✗	~ usually	✓	200
DELETE	✗	✓	rare	204

idempotent = lift button – press 5x → same floor

POST = doorbell – press 5x → rings 5x!

⑥ URL Anatomy – dissect it!



house trick: protocol = vehicle, host = building, port = gate, path = room, query = "knock twice!"

↪ path picks the thing, query filters it

7 The HTTP Request – inside the envelope

1. <i>start line</i>	POST /students HTTP/1.1	← method + URL + version
2. <i>headers</i>	Host: college.com Content-Type: application/json Authorization: Bearer eyJhbG... Cookie: session=abc123	← labels on the envelope
3. <i>blank line</i>	(this empty line separates headers from body – really!)	
4. <i>body</i>	{ "name": "Aman", "course": "BCA" }	← the parcel inside

★ GET & DELETE usually have NO body – just the envelope

7 Headers to Know ↴

Content-Type	"my body is JSON" → application/json — how should the server read this?
Accept	"please REPLY in JSON" — what the client wants back
Authorization	the ID card → Bearer <JWT token> — who is asking?
Cookie	small notes the server gave you earlier, returned every time
Host	which website on this server — one IP, many sites!
User-Agent	"I am Chrome on Windows" — who's knocking

Content-Type = what I'm SENDING

Accept = what I want BACK

X classic bug: POST JSON without Content-Type → req.body = undefined!

8 The HTTP Response – the reply packet

1. *status line* HTTP/1.1 200 OK ← version + code + reason

2. *headers*

Content-Type: application/json
Content-Length: 58
Set-Cookie: session=abc123
Cache-Control: max-age=3600

3. *body*

{ ← the actual data (JSON!)
 "id": 99,
 "name": "Riya",
 "course": "BCA"
}

Why JSON? ★

- ✓ JavaScript Object Notation – Node speaks it natively
- ✓ human-readable, machine-parseable
- ✓ lighter than XML: <name>Riya</name> → "name": "Riya"
- ✓ every language can parse it

read a response in 3 looks

1. *status code* → did it work?
2. *Content-Type* → what came back?
3. *body* → the goods

⑨ Status Codes – the server's mood ★⚡

1xx	"hold on..."	informational – rare in daily life
2xx	"here you go ✓"	success – order delivered
3xx	"go over there →"	redirection – shop moved
4xx	"YOUR fault"	client error – fix your request
5xx	"MY fault"	server error – only the dev can fix it

rhyme: 1 wait • 2 great • 3 relocate • 4 YOUR mistake • 5 server's fate

9 1xx "hold on" & 2xx "here you go"

1xx – informational

100 Continue – "go ahead, send the rest." Like a waiter nodding while you order. Used before big uploads.

101 Switching Protocols – "let's change languages!" HTTP → WebSocket upgrade (chat apps do this).

trick: 1xx = "1 minute..."

2xx – success ✓

200 OK – "done, here it is." GET /students → list. The everyday hero.

201 Created – "new thing made!" POST /students → new admission. ↴
POST should return 201, not 200!

202 Accepted – "got your order, cooking later." Queued jobs, email sending.

204 No Content – "done, nothing to show." Perfect after DELETE.
Empty body!

trick: 2xx = "too good" • 201 = 2-0-WON a new row!

⑨ 3xx – "not here, go there →"

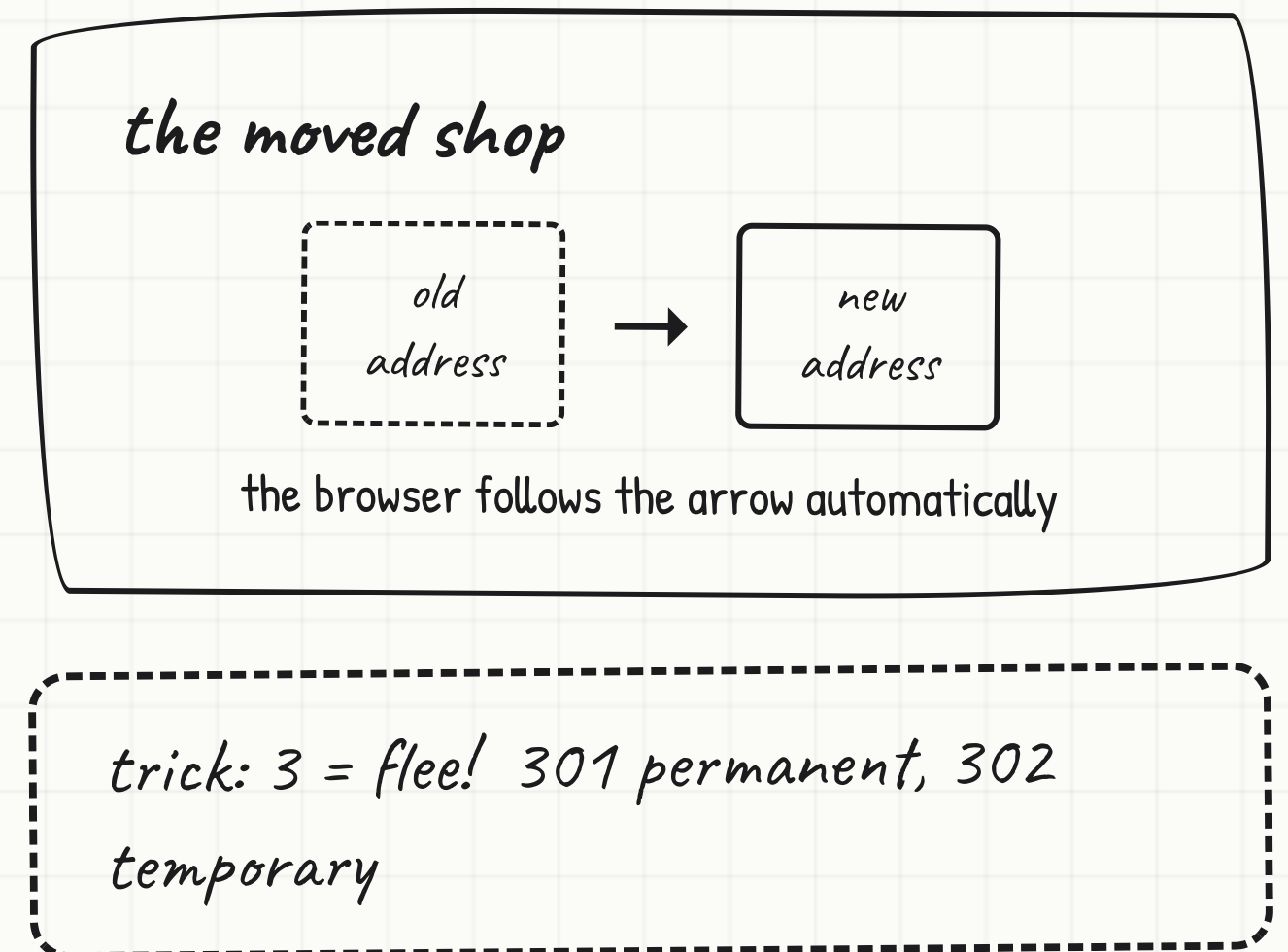
301 Moved Permanently – shop shifted forever, update your address book. old-site.com → new-site.com. SEO juice moves too!

302 Found – temporarily at a different counter today.

304 Not Modified – "your saved copy is still fresh, use it!" Works with ETag. Saves data!

307 Temporary Redirect – like 302, but the method must stay the same (POST stays POST).

308 Permanent Redirect – like 301, method preserved.



⑨ 4xx — "YOU messed up"

400 Bad Request — gibberish form. Broken JSON, missing fields.

403 Forbidden — ID seen, still NO. Logged in $\neq$ allowed! ↴

405 Method Not Allowed — right door, wrong action.

408 Request Timeout — you took too long; the waiter walked away.

410 Gone — existed once, deleted forever. Stronger than 404.

422 Unprocessable Entity — valid JSON, nonsense values: "age": -5.

401 Unauthorized — no ID card! Not logged in. (really: UN-authenticated) ↴

404 Not Found — no such room. GET /studnets (typo!). The celebrity ★

406 Not Acceptable — "you'll only Accept: XML? I only cook JSON."

409 Conflict — two people booked seat 14A. Duplicate email!

415 Unsupported Media Type — a file type the server can't chew.

429 Too Many Requests — "slow down!!" You hammered the API.

401 = who are you? 403 = I know you... still no!

404 = "4-Oh!-4got to check the URL"

⑨ 5xx – "MY fault, sorry"

500 Internal Server Error – kitchen on fire. An uncaught exception in your Node code. Check the server logs, ALWAYS.

501 Not Implemented – "we don't serve that dish yet." Method not built.

502 Bad Gateway – the middleman got garbage. Nginx is fine, but your Node app behind it crashed. ↴

503 Service Unavailable – "closed for maintenance, come later." Overloaded or deploying.

504 Gateway Timeout – the middleman waited... the kitchen never answered. Slow DB query behind a proxy.

the 5-line summary 

1xx → hold on

2xx → here you go ✓

3xx → go there →

4xx → you messed up

5xx → I messed up

trick: 5xx = "Server's fault" – the user can't fix it!

10 REST Naming Rules – write URLs like a pro

<i>X wrong</i>	<i>✓ right</i>
GET /getStudents	GET /students (the verb lives in the method!)
POST /createNewStudent	POST /students
GET /student (singular)	GET /students (always plural!)
GET /Students_List	GET /students (lowercase, kebab-case)
POST /students/delete/99	DELETE /students/99
GET /students/99/get-courses	GET /students/99/courses (nesting = ownership)

★ golden rule: URL = noun (plural) · method = verb

10 Versioning & Consistency

versioning – never break old apps!

`/api/v1/students`

old apps keep working

`/api/v2/students`

new shape ships here

like textbook editions – old-edition students can still study!

safe & idempotent – recap ↴

safe methods: GET (+ HEAD, OPTIONS) – window shopping: look, never touch

idempotent: GET, PUT, DELETE – elevator button

NOT idempotent: POST – doorbell, every press counts!

networks retry failed requests – retrying POST can double-charge a payment!

consistency checklist 

- plural nouns everywhere: /students /courses /orders
- lowercase + kebab-case: /course-modules
- filters in query: /students?year=2&sort=name
- version prefix: /api/v1/...
- right status codes: 201, 204, 404, 422...
- same JSON shape for every error

11 HTTP Facts – history & plumbing

born at CERN

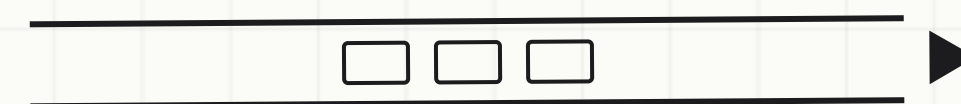
Tim Berners-Lee invented HTTP (+ HTML + the first browser!) in 1989-91 – to share physics papers.

HTTP/1.1 (1997)

added **keep-alive**: reuse one TCP connection for many requests instead of redialling every time. Still everywhere!

HTTP/2 (2015)

multiplexing – many trucks in ONE pipe, at once:



HTTP/3 + QUIC

runs on **UDP**, not TCP! Google's QUIC → faster on flaky mobile networks. YouTube already uses it.

DNS comes FIRST

the internet's phonebook: college.com → 142.250.4.1. No DNS answer = no HTTP at all!

TCP handshake + TLS

SYN → "hello?"

← SYN-ACK "hello! ready"

ACK → "ok, talking now"

then TLS swaps keys (the S in HTTPS) – all BEFORE any HTTP byte moves.

11 HTTP Facts – speed & identity

why browsers cache

fetching again = slow + costly. Cache-Control: max-age=3600 → "keep this for 1 hour, don't ask me again."

ETag = fingerprint

server tags data: ETag "v42". Next time the client asks "still v42?" → server: **304 Not Modified**, zero data resent!

cookies vs sessions vs JWT ↴

cookie = note in your pocket (browser)

session = register at reception (server remembers)

JWT = tamper-proof wristband — server checks the seal, remembers nothing. Cures statelessness!

compression

Accept-Encoding: gzip, br → the server squeezes the response. **Gzip** = classic; **Brotli** = Google's, ~20% smaller. Free speed!



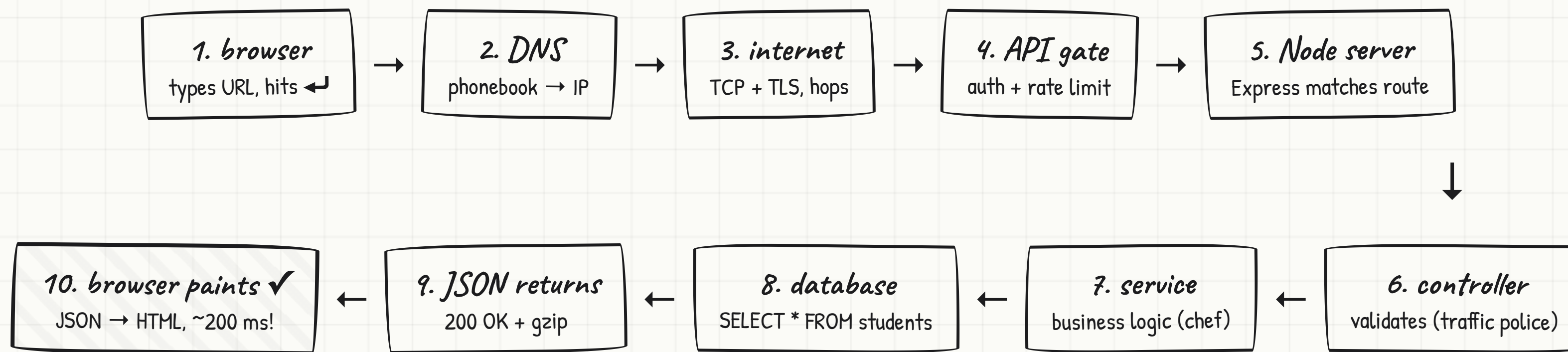
CDN = local kirana

copies of files stored near YOU (Mumbai, Delhi, Chennai edges). Netflix video comes from your city, not California!

remember

HTTP forgets — the token remembers. Cache saves trips; compression shrinks parcels; CDN shortens the road.

12 One Click, Full Journey ↕↕



one breath: browser → DNS → internet → API → Node → controller → service → DB → JSON → browser

errors by stop: DNS fails → "server not found" • gate says no → 401/403/429 • controller rejects → 400/422 • service crashes → 500 • DB slow → 504